```
0000 with IO_EXCEPTIONS;¶¬¬¬¬----------¶package¬¬¬¬TEXT_IO¶is¬¬¬¬¬-------¶¶  type FILE_TYPE is¬limited private;¶¶  type FILE_MODE is¬(IN_FILE,¬OUT_FILE);¶¶
     type COUNT¬¬is range 0 .. INTEGER'LAST;¶  subtype¬POSITIVE_COUNT¬is COUNT range 1 ..¬COUNT'LAST;¶  UNBOUNDED¬¬:constant¬COUNT¬:= 0; -- line and page
     length¶¶¬ -- The upper bound¬for COUNT¬is implementation-defined.¶¶  subtype¬FIELD¬¬is INTEGER range 0 .. INTEGER'LAST;¶  subtype¬NUMBER_BASE¬is INTEG
     ER range 2 .. 16;¶¶¬ -- The upper bound¬for FIELD¬is implementation-defined.¶¶  type TYPE_SET is (LOWER_CASE, UPPER_CASE);¶¶¬ -- File Management¶¶  pr
     ocedure CREATE¬¬( FILE :in out FILE_TYPE;¶¬¬¬  MODE :in FILE_MODE := OUT_FILE;¶¬¬¬  NAME :in STRING := "";¶¬¬¬  FORM :in STRING := ""¶¬¬¬);¶  procedur
     e OPEN¬¬( FILE :in out FILE_TYPE;¶¬¬¬  MODE :in FILE_MODE;¶¬¬¬  NAME :in STRING;¶¬¬¬  FORM :in STRING := ""¶¬¬¬);¶  procedure CLOSE¬¬( FILE :in out FI
     LE_TYPE );¶  procedure DELETE¬¬( FILE :in out FILE_TYPE );¶¶  procedure RESET¬¬( FILE :in out FILE_TYPE; MODE :in FILE_MODE );¶  procedure RESET¬¬( FI
     LE :in out FILE_TYPE );¶¶  function MODE¬¬( FILE :in FILE_TYPE )¬¬return FILE_MODE;¶  function NAME¬¬( FILE :in FILE_TYPE )¬¬return STRING;¶  function
      FORM¬¬( FILE :in FILE_TYPE )¬¬return STRING;¶¶  function IS_OPEN¬¬( FILE :in FILE_TYPE )¬¬return BOOLEAN;¶¶¬ -- Control of default input and output¬f
     iles¶¶  procedure SET_INPUT¬( FILE :in FILE_TYPE );¶  procedure SET_OUTPUT¬( FILE :in FILE_TYPE );¶¶  function STANDARD_INPUT¬¬¬¬return FILE_TYPE;¶
0010 function STANDARD_OUTPUT¬¬¬¬return FILE_TYPE;¶¶  function CURRENT_INPUT¬¬¬¬return FILE_TYPE;¶  function CURRENT_OUTPUT¬¬¬¬return FILE_TYPE;¶¶¬ -- S
     pecification of line and¬page lengths¶¶  procedure SET_LINE_LENGTH¬( FILE :in FILE_TYPE; TO :in COUNT );¶  procedure SET_LINE_LENGTH¬( TO   :in COUNT)
     ;¶¶  procedure SET_PAGE_LENGTH¬( FILE :in FILE_TYPE; TO :in COUNT );¶  procedure SET_PAGE_LENGTH¬( TO   :in COUNT);¶¶  function LINE_LENGTH¬( FILE :in
      FILE_TYPE )¬¬return COUNT;¶  function LINE_LENGTH¬¬¬¬return COUNT;¶¶  function PAGE_LENGTH¬( FILE :in FILE_TYPE )¬¬return COUNT;¶  function PAGE_LEN
     GTH¬¬¬¬return COUNT;¶¶¬ -- Column, Line, and Page Control¶¶  procedure NEW_LINE¬( FILE¬:in FILE_TYPE;¶¬¬¬  SPACING¬:in POSITIVE_COUNT := 1);¶  proced
     ure NEW_LINE¬( SPACING¬:in POSITIVE_COUNT := 1);¶¶  procedure SKIP_LINE¬( FILE¬:in FILE_TYPE;¶¬¬¬  SPACING¬:in POSITIVE_COUNT := 1);¶  procedure SKIP_
     LINE¬( SPACING¬:in POSITIVE_COUNT := 1);¶¶  function END_OF_LINE¬( FILE :in FILE_TYPE)¬¬return BOOLEAN;¶  function END_OF_LINE¬¬¬¬return BOOLEAN;¶¶
     procedure NEW_PAGE¬( FILE :in FILE_TYPE );¶  procedure NEW_PAGE;¶¶  procedure SKIP_PAGE¬( FILE :in FILE_TYPE );¶  procedure SKIP_PAGE;¶¶  function END
     _OF_PAGE¬( FILE :in FILE_TYPE )¬¬return BOOLEAN;¶  function END_OF_PAGE¬¬¬¬return BOOLEAN;¶¶  function END_OF_FILE¬(FILE :in¬FILE_TYPE¬)¬¬return BOOL
     EAN;¶  function END_OF_FILE¬¬¬¬return BOOLEAN;¶¶  procedure SET_COL¬¬(FILE :in¬FILE_TYPE; TO :in POSITIVE_COUNT );¶  procedure SET_COL¬¬(TO   :in¬POS
0020 ITIVE_COUNT );¶¶  procedure SET_LINE¬(FILE :in¬FILE_TYPE; TO :in POSITIVE_COUNT );¶  procedure SET_LINE¬(TO   :in¬POSITIVE_COUNT );¶¶  function COL¬¬(
     FILE :in¬FILE_TYPE¬)¬¬return POSITIVE_COUNT;¶  function COL¬¬¬¬¬¬return POSITIVE_COUNT;¶¶  function LINE¬¬( FILE :in FILE_TYPE )¬¬return POSITIVE_COUN
     T;¶  function LINE¬¬¬¬¬¬return POSITIVE_COUNT;¶¶  function PAGE¬¬(FILE :in¬FILE_TYPE¬)¬¬return POSITIVE_COUNT;¶  function PAGE¬¬¬¬¬¬return POSITIVE_CO
     UNT;¶¶¬ -- Character Input-Output¶¶  procedure GET¬¬( FILE :in FILE_TYPE; ITEM :out CHARACTER );¶  procedure GET¬¬( ITEM :out CHARACTER );¶  procedure
      PUT¬¬( FILE :in FILE_TYPE; ITEM :in CHARACTER );¶  procedure PUT¬¬( ITEM :in CHARACTER );¶¶¬ -- String Input-Output¶¶  procedure GET¬¬( FILE :in FILE
     _TYPE; ITEM :out STRING );¶  procedure GET¬¬( ITEM :out STRING);¶  procedure PUT¬¬( FILE :in FILE_TYPE; ITEM :in STRING );¶  procedure PUT¬¬( ITEM :in
      STRING );¶¶  procedure GET_LINE¬( FILE :in FILE_TYPE;¶¬¬¬  ITEM :out STRING;¶¬¬¬  LAST :out NATURAL¶¬¬¬);¶  procedure GET_LINE¬( ITEM :out STRING;¬
     LAST :out NATURAL¬);¶  procedure PUT_LINE¬( FILE :in FILE_TYPE; ITEM :in STRING );¶  procedure PUT_LINE¬( ITEM :in STRING );¶¶¬ -- Generic package¬for
      Input-Output of¬Integer Types¶¶  generic¶    type NUM is range <>;¶¬¬¬----------¶  package¬¬¬INTEGER_IO¶  is¬¬¬----------¶¶    DEFAULT_WIDTH¬: FIELD¬
     ¬:= NUM'WIDTH;¶    DEFAULT_BASE¬: NUMBER_BASE¬:= 10;¶¶¶    procedure GET¬¬( FILE  :in FILE_TYPE;¶¬¬¬  ITEM  :out NUM;¶¬¬¬  WIDTH :in FIELD := 0¶¬¬¬);¶
0030     procedure GET¬¬( ITEM  :out NUM; WIDTH : in FIELD := 0);¶¶    procedure PUT¬¬( FILE  :in FILE_TYPE;¶¬¬¬  ITEM  :in NUM;¶¬¬¬  WIDTH :in FIELD¬¬:= D
     EFAULT_WIDTH;¶¬¬¬  BASE  :in NUMBER_BASE¬:= DEFAULT_BASE¶¬¬¬);¶    procedure PUT¬¬( ITEM  :in NUM;¶¬¬¬  WIDTH :in FIELD¬¬:= DEFAULT_WIDTH;¶¬¬¬  BASE
     :in NUMBER_BASE¬:= DEFAULT_BASE¶¬¬¬);¶    procedure GET¬¬( FROM :in STRING;¶¬¬¬  ITEM :out NUM;¶¬¬¬  LAST :out POSITIVE¶¬¬¬);¶    procedure PUT¬¬( TO
       :out STRING;¶¬¬¬  ITEM :in NUM;¶¬¬¬  BASE :in NUMBER_BASE¬:= DEFAULT_BASE¶¬¬¬);¶¶  end INTEGER_IO;¶¶¬ -- Generic package¬for Input-Output of¬Real Ty
     pes¶¶  generic¶    type NUM is digits <>;¶¬¬¬----------¶  package¬¬¬FLOAT_IO¶  is¬¬¬---------¶¶    DEFAULT_FORE¬¬: FIELD¬¬:= 2;¶    DEFAULT_AFT¬¬: FIELD¬
     ¬:= NUM'DIGITS-1;¶    DEFAULT_EXP¬¬: FIELD¬¬:= 3;¶¶¶    procedure GET¬¬( FILE  :in FILE_TYPE;¶¬¬¬  ITEM  :out NUM;¶¬¬¬  WIDTH :in FIELD¬¬:= 0¶¬¬¬);¶
       procedure GET¬¬( ITEM  :out NUM; WIDTH :in FIELD := 0);¶¶    procedure PUT¬¬( FILE :in FILE_TYPE;¶¬¬¬  ITEM :in NUM;¶¬¬¬  FORE :in FIELD¬¬:= DEFAULT
     _FORE;¶¬¬¬  AFT  :in FIELD¬¬:= DEFAULT_AFT;¶¬¬¬  EXP  :in FIELD¬¬:= DEFAULT_EXP¶¬¬¬);¶    procedure PUT¬¬( ITEM :in NUM;¶¬¬¬  FORE :in FIELD¬¬:= DEFAU
     LT_FORE;¶¬¬¬  AFT  :in FIELD¬¬:= DEFAULT_AFT;¶¬¬¬  EXP  :in FIELD¬¬:= DEFAULT_EXP¶¬¬¬);¶    procedure GET¬¬( FROM :in STRING;¶¬¬¬  ITEM :out NUM;¶¬¬¬
      LAST :out POSITIVE¶¬¬¬);¶    procedure PUT¬¬( TO   :out STRING;¶¬¬¬  ITEM :in NUM;¶¬¬¬  AFT  :in FIELD¬¬:= DEFAULT_AFT;¶¬¬¬  EXP  :in INTEGER¬¬:= DEF
0040 AULT_EXP¶¬¬¬);¶¬---------¶  end¬FLOAT_IO;¶¬---------¶¶  generic¶    type NUM is delta <>;¶¬¬¬---------¶  package¬¬¬FIXED_IO¶  is¬¬¬----------¶¶    DEFAULT_
     FORE¬¬: FIELD¬¬:= NUM'FORE;¶    DEFAULT_AFT¬¬: FIELD¬¬:= NUM'AFT;¶    DEFAULT_EXP¬¬: FIELD¬¬:= 0;¶¶¶    procedure GET¬¬( FILE  :in FILE_TYPE;¶¬¬¬  ITE
     M  :out NUM;¶¬¬¬  WIDTH :in FIELD¬¬:= 0¶¬¬¬);¶    procedure GET¬¬( ITEM  :out NUM; WIDTH :in FIELD := 0 );¶¶    procedure PUT¬¬( FILE :in FILE_TYPE;¶¬
     ¬¬  ITEM :in NUM;¶¬¬¬  FORE :in FIELD¬¬:= DEFAULT_FORE;¶¬¬¬  AFT  :in FIELD¬¬:= DEFAULT_AFT;¶¬¬¬  EXP  :in FIELD¬¬:= DEFAULT_EXP¶¬¬¬);¶    procedure P
     UT¬¬( ITEM :in NUM;¶¬¬¬  FORE :in FIELD¬¬:= DEFAULT_FORE;¶¬¬¬  AFT  :in FIELD¬¬:= DEFAULT_AFT;¶¬¬¬  EXP  :in FIELD¬¬:= DEFAULT_EXP¶¬¬¬);¶    procedure
      GET¬¬( FROM :in STRING;¶¬¬¬  ITEM :out NUM;¶¬¬¬  LAST :out POSITIVE¶¬¬¬);¶    procedure PUT¬¬( TO   :out STRING;¶¬¬¬  ITEM :in NUM;¶¬¬¬  AFT  :in FIE
     LD¬¬:= DEFAULT_AFT;¶¬¬¬  EXP  :in INTEGER¬¬:= DEFAULT_EXP¶¬¬¬);¶¬---------¶  end¬FIXED_IO;¶¬---------¶¶¬ -- Generic package¬for Input-Output of¬Enumerat
     ion types¶¶  generic¶    type ENUM is (<>);¶¬¬¬----------¶  package¬¬¬ENUMERATION_IO¶  is¬¬¬----------¶¶    DEFAULT_WIDTH¬: FIELD := 0;¶    DE
     FAULT_SETTING¬: TYPE_SET := UPPER_CASE;¶¶¶    procedure GET¬¬( FILE :in FILE_TYPE; ITEM :out ENUM);¶    procedure GET¬¬( ITEM :out ENUM);¶¶    procedu
     re PUT¬¬( FILE  :in FILE_TYPE;¶¬¬¬  ITEM  :in ENUM;¶¬¬¬  WIDTH :in FIELD¬¬:= DEFAULT_WIDTH;¶¬¬¬  SET   :in TYPE_SET¬:= DEFAULT_SETTING¶¬¬¬);¶    proce

0050 dure PUT¬¬( ITEM  :in ENUM;¶¬¬¬  WIDTH :in FIELD¬¬:= DEFAULT_WIDTH;¶¬¬¬  SET   :in TYPE_SET¬:= DEFAULT_SETTING¶¬¬¬);¶    procedure GET¬¬( FROM :in STR
     ING;¶¬¬¬  ITEM :out ENUM;¶¬¬¬  LAST :out POSITIVE¶¬¬¬);¶    procedure PUT¬¬( TO   :out STRING;¶¬¬¬  ITEM :in ENUM;¶¬¬¬  SET  :in TYPE_SET¬¬:= DEFAULT_
     SETTING¶¬¬¬);¶¬---------¶  end¬ENUMERATION_IO;¶¬---------¶¶¬ -- Exceptions¶¶¬ STATUS_ERROR   : exception renames IO_EXCEPTIONS.STATUS_ERROR;
     ¶¬ MODE_ERROR     : exception renames IO_EXCEPTIONS.MODE_ERROR;¶¬ NAME_ERROR     : exception renames IO_EXCEPTIONS.NAME_ERROR;¶¬ USE_ERROR      : exce
     ption renames IO_EXCEPTIONS.USE_ERROR;¶¬ DEVICE_ERROR   : exception renames IO_EXCEPTIONS.DEVICE_ERROR;¶¬ END_ERROR      : exception renames IO_EXCEPT
     IONS.END_ERROR;¶¬ DATA_ERROR     : exception renames IO_EXCEPTIONS.DATA_ERROR;¶¬ LAYOUT_ERROR   : exception renames IO_EXCEPTIONS.LAYOUT_ERROR;¶¶priva
     te¶¶  subtype¬FILE_NAME_BUFFER¬is STRING( 1 .. 256¬);¶¶  type FILE_TYPE¬¬is record¶¬¬¬  ID¬¬: INTEGER¬¬:= -1;¶¬¬¬  NAME¬¬: FILE_NAME_BUFFER;¶¬¬¬  NAME
     _LEN¬: POSITIVE;¶¬¬¬  IS_OPENED¬: BOOLEAN¬¬:= FALSE;¶¬¬¬  MODE¬¬: FILE_MODE;¶¬¬¬  PAGE_LENGTH,¶¬¬¬  LINE_LENGTH,¶¬¬¬  PAGE,¶¬¬¬  LINE,¶¬¬¬  COL¬¬: POS
     ITIVE_COUNT;¶¬¬¬  IS_DEFAULT_IO¬: BOOLEAN¬¬:= FALSE;¶¬¬¬  LOOK_AHEAD¬: CHARACTER¬:= ASCII.NUL;¶¬¬¬  HAS_LOOK_AHEAD¬: BOOLEAN¬¬:= FALSE;¶¬¬¬  AT_END_OF
     _FILE¬: BOOLEAN¬¬:= FALSE;¶¬¬¬end record;¶¶¬--------¶ end¬TEXT_IO;¶¬--------¶
```